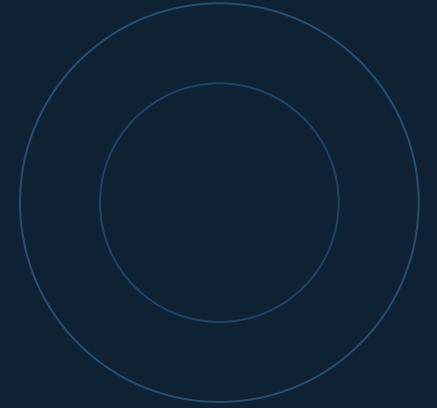


Optimal Emergency Blood Supply Distribution



A C++17 decision engine combining shortest-path routing, greedy triage, maximum flow and dynamic programming

SPEAKER A

Md. Iftakhar Alam

230041231 · Problem & selection

SPEAKER B

Huzaifa Sani

230041263 · Design & algorithms

SPEAKER C

Oumar Ibrahim

230041247 · Demo, results & testing

How this presentation runs

SPEAKER A

PART ONE · SPEAKER A ·

The problem and why we chose it

1. The emergency scenario
2. Why a nearest-hospital rule fails
3. The three proposals compared
4. Selection criteria and trade-off

PART TWO · SPEAKER B ·

How the system is built

1. The five-stage pipeline
2. Graph and flow-network model
3. Input format
4. Code: routing, triage, flow, DP
5. Two decisions that matter

PART THREE · SPEAKER C ·

Running it, and what it produces

1. Live demo in the terminal
2. Allocation and dispatch output
3. Complexity and structures
4. Verification suite
5. Limitations and closing

Several hospitals call at once. Supply does not stretch.

SPEAKER A

A building collapse, a bus crash, a cyclone. Within minutes a blood bank coordinator is holding four simultaneous requests, each naming a different group, a different quantity and a different deadline.

CONSTRAINT 1

Demand is uneven

Different groups, urgencies and deadlines, all arriving at the same moment.

CONSTRAINT 2

Supply is scarce

Each bank holds limited stock per group. Rare groups run out first.

CONSTRAINT 3

Transport is limited

Vehicles have carrying limits; roads take time. A plan must be loadable.

CONSTRAINT 4

Time is absolute

Blood arriving after the deadline has no clinical value at all.

WHY NEAREST-FIRST FAILS

The closest hospital may not hold the most critical patients, and the closest bank may not stock the right group.

THE DECISION TO MAKE

Which bank serves which request, how many units of which group, along which route, on which vehicle.

Why we chose this problem

SPEAKER A

Member	Proposed problem	Algorithms	Outcome
Md. Iftakhar Alam	Emergency blood supply distribution	Dijkstra · max-flow · greedy · DP	Selected

Four load-bearing algorithms

Remove any one and the system stops working, not merely degrades.

Stages genuinely chain

Each stage's output constrains the next — CEP attribute P7, visible in code.

No external data needed

Runs on a scenario file we author — no live traffic API or map licence.

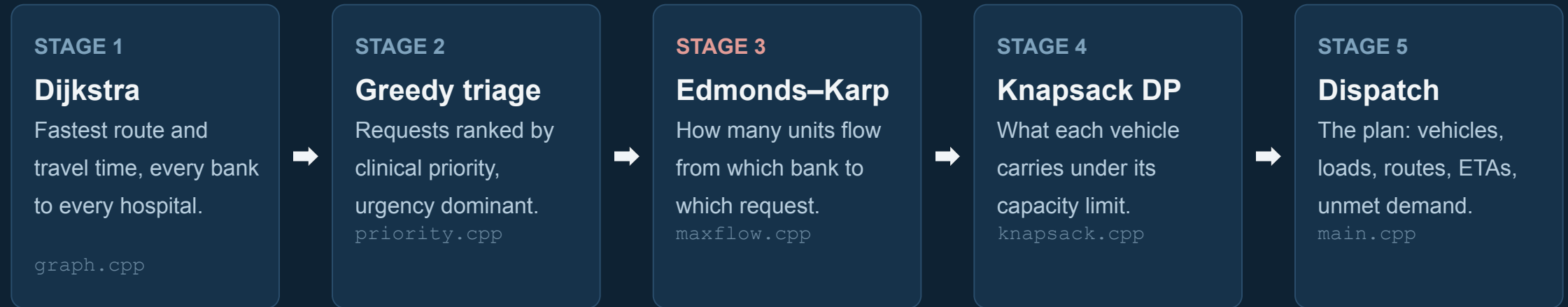
Output is a decision

A dispatch plan a reader can verify by hand, not a score.

The argument against: four algorithms is more work than a single-algorithm problem. We accepted that cost because the modular pipeline made the work divisible between three people.

Five stages. Each one feeds the next.

SPEAKER A



WHY SPLIT IT AT ALL

Solved jointly this is a vehicle routing problem with time windows — **NP-hard**. Solved as a pipeline, each stage is polynomial with a known guarantee for its own sub-problem.

HOW THE CHAINING WORKS

Stage 1's travel times become deadline feasibility tests inside stage 3. Stage 3's allocations become the items stage 4 packs.

Two models: a road graph and a flow network

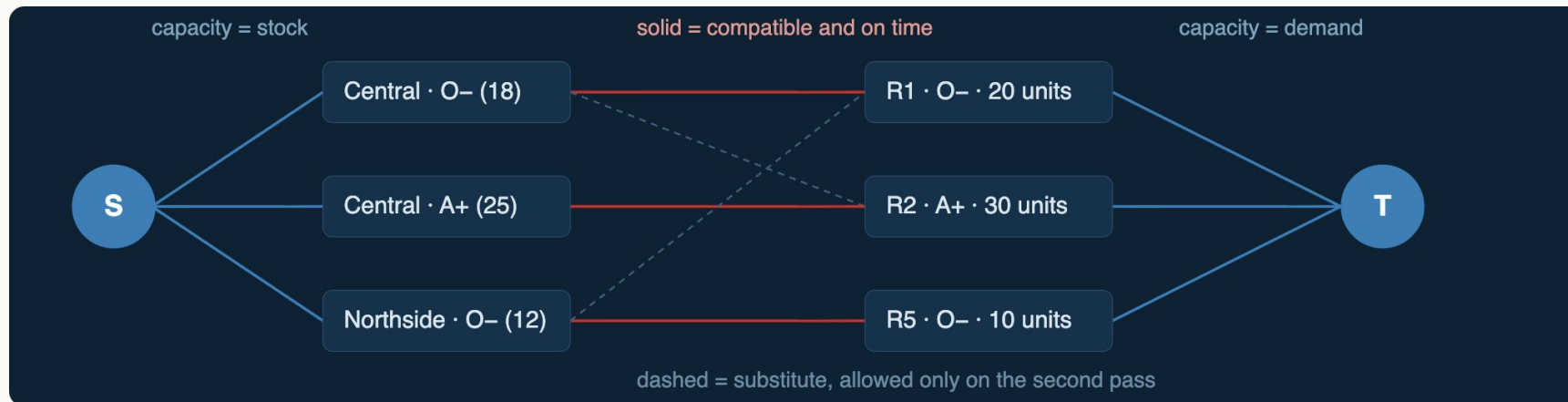
SPEAKER B

The city as a weighted graph

$G = (V, E)$ — nodes are banks, hospitals and junctions; every edge carries a non-negative travel time in minutes. Stage 1 runs on this.

Allocation as a flow network

A second, separate graph built per urgency tier. Supply enters at the source, demand drains at the sink. Stage 3 runs on this.



An edge exists only if the group is transfusable **and** the bank can reach that hospital before the deadline. Medical constraints are absolute, so they are structure — never a cost the flow can trade away.

Input: one plain-text scenario file

SPEAKER B

```
data/sample_scenario.txt
NODES 14
0 Central_Blood_Bank BANK
2 City_General_Hospital HOSPITAL
5 Junction_J1 JUNCTION

EDGES 20
0 5 4 road, 4 minutes, two-way

BANKS 2
0 5 bank at node 0, 5 groups
O- 18
O+ 40

REQUESTS 5
2 O- 20 CRITICAL 9 30
4 O- 15 HIGH 6 45

VEHICLES 3
```

Decoding one request line

2 O- 20 CRITICAL 9 30

Hospital at node 2 · O-negative · 20 units · CRITICAL urgency · 9 critical patients · needed within 30 minutes

Accepted values

Groups O- O+ A- A+ B- B+ AB- AB+

Urgency LOW MEDIUM HIGH CRITICAL

Names use underscores; # starts a comment

Errors name their section

REQUESTS: unknown blood group AB

The count after each keyword must match the rows that follow — the commonest editing mistake.

Run with no argument and a built-in scenario loads instead, so the program always produces output — useful for a demo that must not fail.

Stages 1–2 Routing and triage

graph.cpp · priority.cpp

SPEAKER B

Dijkstra — relaxation over a binary heap

```
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();

    // lazy deletion: an outdated entry
    if (d > sp.dist[u]) continue;

    for (const Road& e : adj[u]) {
        double nd = d + e.weight;
        if (nd < sp.dist[e.to]) {
            sp.dist[e.to] = nd;
            sp.parent[e.to] = u; // for the route
            pq.push({nd, e.to});
        }
    }
}
```

Travel times are never negative — exactly Dijkstra's precondition, so there is no reason to pay for Bellman–Ford. The parent array is what lets us print the actual road route, not just a number.

Triage — a score that is lexicographic in urgency

```
double tightness =
    max(0.0, w.deadlineBase - r.deadline);

double secondary =
    w.patientWeight * r.criticalPatients
    + w.deadlineWeight * tightness;

// the cap is what keeps urgency dominant
secondary = min(secondary, w.secondaryCap);

return w.urgencyWeight * r.urgency
```

Urgency weight is 10,000; the secondary terms are capped at 999. No number of lower-urgency patients and no deadline however tight can lift a HIGH request above a CRITICAL one. This encodes triage policy, not throughput.

Stages 3–4 Allocation and loading

SPEAKER B

maxflow.cpp · knapsack.cpp

Feasibility encoded as edge existence

```
// 1. group must be transfusable
if (!compatible(x.group, r.group)) continue;

// 2. exact pass: patient's own group only
if (exactOnly && x.group != r.group) continue;

// 3. must arrive before the deadline
double t = sp[x.bank].dist[r.hospitalNode];
if (t == INF_TIME || t > r.deadline) continue;

int cap = min(stock[x.bank][x.group], remaining);
links.push_back({ ..., mf.addEdge(
    x.node, reqNode[i], cap) });
```

Edmonds–Karp picks augmenting paths by BFS, bounding augmentations at $O(V \cdot E)$ and giving $O(V \cdot E^2)$ independent of capacity magnitudes — which matters because unit counts are data-dependent.

Knapsack — recurrence and traceback

```
for (int i = 1; i <= n; ++i)
    for (int c = 0; c <= capacity; ++c) {
        dp[i][c] = dp[i-1][c]; // skip item i
        if (wi <= c) {
            double take = dp[i-1][c-wi] + vi;
            if (take > dp[i][c]) dp[i][c] = take;
        }
    }

// walk back: WHICH items were taken
for (int i = n; i >= 1; --i)
    if (dp[i][c] != dp[i-1][c]) {
        chosen.push_back(i-1); c -= weights[i-1];
    }
```

Weight is units, value is priority-weighted units. The DP value alone does not say what to load, so the traceback is not optional — the selected items *are* the deliverable.

THE TWO DECISIONS A NAIVE IMPLEMENTATION GETS WRONG

SPEAKER B

DECISION ONE

Max-flow runs in urgency tiers

A single global max-flow maximises *total units delivered* and will starve one critical request to satisfy two routine ones. The flow has no notion that one unit matters more than another.

CRITICAL first, then HIGH, residual stock carried forward. This is what makes the triage ranking actually bind.

DECISION TWO

Exact matches before substitutes

Otherwise the network treats every compatible unit as interchangeable and spends scarce O-negative on a patient who could have received A-positive.

Two passes per tier: the patient's own group first, compatible substitutes only on what remains.

Both are cases where the textbook algorithm gives a mathematically optimal answer to the *wrong question*. The correction lies in how the problem is encoded, not in the algorithm.

Running the system

SPEAKER C

TERMINAL

```
$ make
```

```
builds ./bloodflow, no dependencies
```

```
$ make test
```

```
14 checks, all passing
```

```
$ ./bloodflow
```

```
built-in scenario, five stages
```

```
$ ./bloodflow data/my_scenario.txt
```

```
our own edited scenario
```

1 · Build and verify

A clean build and a passing suite on screen before anything else. It establishes that the rest is real.

2 · Run the sample

Scroll through all five stages, naming each as it passes. End on the summary table.

3 · Change the scenario, re-run

Cut O-negative stock at Central and re-run. Watch a HIGH request fall to UNMET while both CRITICAL requests stay satisfied.

4 · Tighten a deadline

Set a deadline below the achievable travel time. The request receives nothing, because the edge is never created.

Steps 3 and 4 are the demonstration that matters: they show the design decisions producing visibly different outcomes, rather than asserting them on a slide.

Output: routing, triage, allocation

SPEAKER C

Stage 1 · shortest routes

```
Central    -> City General  13.0 min
  Central -> J1 -> J3 -> City Gen.
Central    -> Riverside    22.0 min
Central    -> Metro Trauma 23.0 min
Northside  -> City General  16.0 min
Northside  -> Riverside    15.0 min
Northside  -> Metro Trauma 20.0 min
```

Northside reaches Riverside faster than Central, though Central is the larger facility.

Stage 2 · triage ranking

Rank	Req	Urgency	Crit	Score
1	R1	CRITICAL	9	30135
2	R5	CRITICAL	7	30115
3	R3	HIGH	6	20105
4	R4	HIGH	3	20075
5	R2	MEDIUM	2	10040

Tiers are 10,000 apart. Patient count and deadline only order requests inside a tier.

Stage 3 · allocation, tier by tier

```
Tier CRITICAL  requests: 2
  Central    -> R1  18 units O- (exact)
  Northside  -> R1   2 units O- (exact)
  Northside  -> R5  10 units O- (exact)
  subtotal: 30 units (30 exact, 0 subst.)

Tier HIGH      requests: 2
  Central    -> R4  10 units B+ (exact)
  Northside  -> R4   2 units B+ (exact)
  subtotal: 12 units

Tier MEDIUM   requests: 1
  Central    -> R2  25 units A+ (exact)
```

Every allocation is an exact group match — the conservation pass working. R1 is served by two banks at once.

Nothing forces a request onto a single supplier. Where splitting across two banks is the feasible answer, the flow finds it.

Output: the dispatch plan

SPEAKER C

```
V1 (Central) from Central Blood Bank (23/25 units)
  R1 City General 18 units O- ETA 13.0 (deadline 30.0)
    route: Central -> J1 -> J3 -> City General
  R2 Riverside    5 units A+ ETA 22.0 (deadline 90.0)

V2 (Central) from Central Blood Bank (10/20 units)
  R4 City General 10 units B+ ETA 13.0 (deadline 60.0)

V3 (Northside) from Northside (19/30 units)
  R1 City General  2 units O- ETA 16.0 (deadline 30.0)
  R5 Riverside    10 units O- ETA 15.0 (deadline 40.0)
  R4 City General  2 units B+ ETA 16.0 (deadline 60.0)

Allocated but awaiting a second trip: 25 units
```

Req	Requested	Allocated	Status
R1	20	20	fully met
R2	30	30	fully met
R3	15	0	UNMET
R4	12	12	fully met
R5	10	10	fully met
Total 87 · allocated 72 · fulfilment 82.8%			

R3 gets nothing — and that is correct

The city holds 30 units of O-negative against 45 units of demand. Both CRITICAL requests consumed it. An unrestricted flow could have satisfied R3 by shorting R1 or R5 — more units delivered, worse clinical outcome.

The reported shortfall is the useful output: it tells the coordinator to call for an outside transfer.

Data structures and complexity

SPEAKER C

Structure	Used for	Why this one
Adjacency list	Road network	Road networks are sparse; a matrix would waste $O(V^2)$ space
Binary min-heap	Dijkstra frontier	$O(\log V)$ extract-min; lazy deletion removes the need for decrease-key
Paired edge array, reverse at i^1	Flow network	Constant-time reverse edge, needed on every augmentation
Fixed array of 8 unit counts	Blood bank stock	Blood group is a small fixed enum, so indexing beats a hash map
Static 8×8 boolean matrix	ABO/Rh compatibility	Constant-time lookup of a medical rule that never changes at runtime
2-D DP table with traceback	Vehicle loading	The selected items, not just the optimal value, are the actual output

DIJKSTRA · per bank
 $O((V+E) \log V)$

TRIAGE SORT
 $O(R \log R)$

EDMONDS-KARP · per pass
 $O(V \cdot E^2)$

KNAPSACK · per vehicle
 $O(n \cdot C)$

Verification: 14 checks against hand-computed answers

SPEAKER C

```
$ make test
```

```
PASS Dijkstra: distance 0->3 is 2
PASS Dijkstra: path 0->3 is 0-2-3
PASS Dijkstra: isolated node unreachable
PASS Edmonds-Karp: max flow is 5
PASS Edmonds-Karp: bottleneck caps flow
PASS Knapsack: optimum value is 220
PASS Knapsack: uses full 50 capacity
PASS Knapsack: oversized item rejected
PASS Compatibility: O- donates to AB+
PASS Compatibility: A+ cannot give O-
PASS Triage: CRITICAL outranks MEDIUM
```

```
All checks passed. Failures: 0
```

One test failed on its first run — and it was right

The triage check exposed a real defect: the original weights were commensurate, so fifty medium-urgency patients outranked a critical request.

The scoring function was rewritten to the capped lexicographic form shown earlier. **The tests changed the design rather than merely confirming it.**

The parser is verified separately: the scenario file produces output identical to the built-in scenario, confirming both input paths agree.

What the system does not do

SPEAKER C

Not globally optimal

Each stage is optimal for its own sub-problem, but composing four locally optimal decisions does not give a globally optimal plan. A joint formulation would be an integer program.

Travel times are static

Real disaster response involves roads closing mid-operation. The architecture supports re-running stage 1 and re-planning, but the driver does not simulate it.

One trip per vehicle

Leftover allocations are reported as awaiting a second trip rather than scheduled into one. Multi-trip routing would sit above stage 4.

Clinical model stops at ABO/Rh

Shelf life, cold-chain duration and full crossmatching are not represented, and tie-breaking inside a tier is deterministic but not clinically motivated.

WHAT THIS IS

The decision engine — the part that does the thinking, and the part that is a CSE 4403 problem.

WHAT A DEPLOYED VERSION ADDS

A web form instead of a text file, a live map instead of typed edge weights, and plans pushed to drivers' phones.

Four algorithms, composed so each one constrains the next

The system turns simultaneous emergency requests into an actionable plan — which bank sends what to whom, along which route, on which vehicle, arriving when — and states clearly what it cannot deliver.

REPOSITORY

<https://github.com/aether1731/build-and-run-ood-supply-system>

DEMONSTRATION VIDEO

will be given soon

BUILD AND RUN

`make · make test · make run`